

TECS SUBMISSION · EMERGING WORLD MODELS

Transformations and trajectories of a content-addressed state machine

The Emerging World Model HLLSet architecture: a state layer where the state is a set, every transition is a lattice operation, and the trajectory is a rotating coordinate system you can read.

BOOLEAN LATTICE · GF(2) RING · CONTENT-ADDRESSED · SIDE-CAR MEMORY

ALEX MYLNIKOV · ALEXANDR MYLNIKOV JR. · DEEPENDRA KUMAR

github.com/alexmy21/ewm-state-machine



Models are functions. Nothing is kept.

A language model, a vision model, a planner — each is a function: input in, output out, nothing kept. What we call its "memory" is a text buffer or a key-value cache, **rebuilt from scratch for every request**.

So the system has no **state** in the control-theoretic sense: no value that is advanced, committed and recovered; no record of what was retained, what departed, and what is new.

Rebuilding context from scratch is where much of the **cost** — and most of the **failure modes** — of long-horizon systems live: lost continuity, silent drift, unrecoverable crashes, un-auditable behavior.

The question this paper answers: can any token-producing model be given a state that is **cheap to advance**, **exact to compare**, and **safe to replay** — without touching the model?

WHAT "MEMORY" IS TODAY

context window

a text buffer, re-derived per request — a buffer, not a state

KV cache

an inference accelerator — discarded, never committed or recovered

retrieval (RAG)

an attached lookup — it finds documents, it does not hold a state

WHAT IS MISSING

A value that is **advanced** (transition), **committed** (durable record) and **recovered** (replay) — with what-departed / what-stayed / what-is-new read off exactly.

The state is a content-addressed set, not a tensor

The whole design follows from one decision: represent every state as a **set of bits over a fixed plane**, and name each set by its own content. A token stream is measured into an HLLSet — a fixed-length bit vector whose set bits are the sites the tokens hashed into.

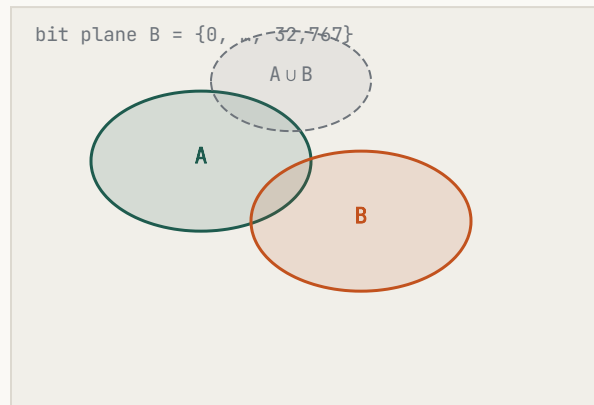
Subsets of one common plane form a **Boolean lattice**: order $\subseteq$, join $\cup$, meet $\cap$, ring addition Δ (symmetric difference).

Every state and every derived quantity — unions, intersections, differences, novelty, residuals — is a node of **one** lattice. Nothing new is ever created: a computation names a lattice expression and **discovers** the node.

Two paths that name the same expression reach the same node. That is the operational meaning of *one universe*.

Every node obeys **IICA**: Idempotent · Immutable · Content-Addressed. A node is literally the digest of its bits — two computations that reach the same bits reach the same state.

ONE LATTICE — EVERYTHING IS A NODE



States A, B and their union coexist as nodes of the same lattice — discovered by expression, addressed by content.

THE ALGEBRA IS PLAIN SET THEORY

$\cup$ join (OR) · $\cap$ meet (AND)
 Δ ring addition (XOR) · \ difference

An HLLSet: the sites a token collection marks

An **HLLSet** is a compact sketch of a token collection. It stores no tokens — only a fixed-length bit vector whose set bits are the **measurement sites** the tokens hashed into.

The geometry is **soldered**: precision P fixes the register count $M = 2^P$; each register holds b bits. A token is hashed once and decomposed into a register index and a trailing-zero state.

```
reg(t) = h(t) mod M
tz(t)  = clamp(trailing_zeros(h(t) » P), 0, b-1)
bit(t) = reg(t) · b + tz(t)    →    H(A) = ∪_{t ∈ A} {bit(t)}
```

With the defaults $P = 10$, $b = 32$, the plane has $M \cdot b = 32,768$ bits — 4 KB packed, stored as a Roaring bitmap. A collection is represented by the sites it marks, never by the tokens themselves.

The bit vector deliberately forgets provenance — which token or which scheme set a bit is not recorded.

ONE TOKEN → ONE BIT ADDRESS

token $t \rightarrow$ hash $h(t)$

↓ split the hash

reg = $h \bmod 1024$
→ which register

tz = trailing zeros
→ which bit inside

↓ one site on the plane

THE 32,768-BIT PLANE (DETAIL)



DEFAULT GEOMETRY

$P = 10$ — precision, soldered

$M = 2^{10} = 1,024$ — registers

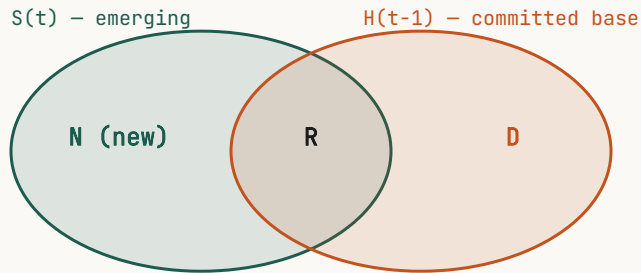
$b = 32$ — bits per register

$M \cdot b = 32,768$ — sites = 4 KB packed

One equation: every term is an HLLSet

$$H(t) = (S(t), H(t-1), D, R, N)$$

THE TEMPORAL SIGNATURE OF A TRANSITION



N, R, D partition the union of the two states into three disjoint pieces — what the new turn brings, what it keeps, what it drops.

THE FIVE TERMS, AS LATTICE OPERATIONS

S(t) — current emerging state (work in progress)

advanced by union each turn: $S(t) \leftarrow S(t) \cup H_{\text{turn}}$

H(t-1) — previous committed state (the base)

the tip of the commit DAG; immutable, content-addressed

N = S(t) \ H(t-1) · R = S(t) ∩ H(t-1) · D = H(t-1) \ S(t)

new · retained · departed — computed per transition

A commit records H(t) and advances the tip atomically; a no-change pass commits nothing. The decomposition is the per-transition reading that all later frames build on.

Four ideas carry the machine

1

One universe

Every HLLSet is a node of the same Boolean lattice over the bit plane. Nothing new is created; a computation names a lattice expression and discovers the node.

2

The morphism is the contract (IICA)

The token $\rightarrow$ bit connection is a chosen measurement μ . Only IICA is required — idempotent, immutable, content-addressed — and any IICA algorithm is interchangeable. Two instances appear here: the flat projection μ_{flat} (a join homomorphism) and windowed n-gram ingestion μ_{ng} (inclusion-isotonic only).

3

The fiber is global

The same bit in any HLLSet points to the same lookup-table fiber: materialization is a property of the bit address, never of the state that contains it — exact on the builders, probabilistic on the extras.

4

Decompositions are frames (readings)

D/R/N, joined perceptrons, the ring basis — each is a named set of dimensions through which a node is read. A frame ϕ_D maps a node to coordinates, and $\phi_D(X(t))$ over time is the state trajectory.

The first two ideas make the state safe to share; the third makes it readable; the fourth makes it **legible over time** — the trajectory is a choice of lens, not a change to the state.

S(t) is the state in a stateless system

IICA — Idempotence · Immutability · Content Addressability — removes the contradiction: S(t) is an **immutable, content-addressed value**, so it is safe to share and safe to rebuild.

The processing unit ([UM]) is **stateless and disposable**: it reads the tip, processes incoming tokens, and proposes the next state. S(t) and H(t-1) live **outside** the [UM] — in a shareable cache (ewm-app::StateCache) backed by a persistent store (ewm-git). A commit is the atomic advance of the tip.

Recovery is a pop, not a rebuild. Drop the broken [UM] and its cache, restore from the tip, continue with a fresh unit. Uncommitted work is reprocessed — harmless by idempotence.

Safe-replay principle: because ingestion is idempotent and the join is union, reprocessing an uncommitted turn after a crash yields the same node as processing it once. The committed stack is never mutated.

persistent store · ewm-git
the committed stack — a content-addressed DAG; head = tip

load ↑ · commit ↓

shared cache · StateCache
S(t) working set + H(t-1)
materialized — a pure value,
shareable between units

read ↑ · propose ↓

[UM] · ewm-app
stateless, disposable — owns
only the store handle

One data structure over three locations. Caching and persistence are separated by content addressing: the cache is rebuildable from the tip, and a crash costs only the uncommitted turn.

Two layers: a foundation that knows no turns

01 FOUNDATION

hllset-core — the HLLSet bit plane (a Roaring bitmap) and its set algebra · **hllset-lut** — the LUT lattice: token interpretation, content-addressed

hllset-morphisms — ingest / materialize + default API · **hllset-contracts** — soldered invariants: P=10, M=1024, b=32, MurmurHash3

hllset-cid — SHA-1 CIDs · **hllset-storage** — memory + sled persistence
vocabulary-agnostic — no notion of a "turn" or a "world"

02 STATE LAYER

ewm-git — the committed stack: commit DAG, tip, recovery · **ewm-app** — the stateless [UM] harness + shared StateCache

context-tree — Merkle tree over the working set (S(t) presentation) · **ewm-cache** — Arrow-backed extended cache

ewm-boolring — the GF(2) ring: BoolWindow + RingStats · **ewm-ops** — content-addressed operational graph (value / program lattice)

ewm-sm-explore — read-only explorer of the three-layer state

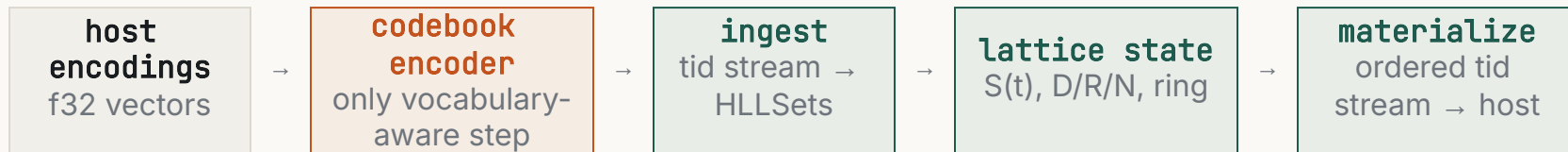
where the machine lives

03 HOSTS

ewm-scene — LLM ↔ HLLSet side-car helper (direct morphisms) · **ewm-flux-host** — side-car host adapter for a rectified-flow sampler

the host is never modified — the state layer attaches beside it

One vocabulary-aware step; everything below is generic



The encoder quantizes host encodings to a shared codebook and emits `tid{n}` tokens. Everything below it is **vocabulary-agnostic**: any stream that can be tokenized can drive the same state machine.

That is what makes the layer a **side-car** rather than a model: it does not interpret content — it accumulates and reads structure.

An LLM, an OCR encoder, a vision model, a planner — the host is left untouched; the state layer attaches beside it.

WHERE EACH STEP LIVES

encode → **CodebookEncoder**

per-host, the only custom piece

ingest → **hllset-morphisms**

the IICA measurement μ

state → **ewm-app · ewm-git · ewm-boolring**

working set, commit stack, ring basis

materialize → **LUT-first inverse read**

bits lowered back to tokens for the host, on demand

One turn: six steps, each exact

A turn is the unit of transformation. Given a token list, every step below is a lattice operation, a Gaussian step, or a commit.

- | | | |
|---|--------------------|---|
| 1 | encode | Host encodings are quantized to a tid stream by the shared codebook encoder — the only vocabulary-aware step. |
| 2 | ingest | The stream is measured by the IICA morphism; the turn's current projection is its original H _T . |
| 3 | join | $S(t) \leftarrow S(t) \cup H_T$. Union is idempotent, commutative, associative — independent turns accumulate without coordination. |
| 4 | ring push | The original enters the GF(2) ring: one Gaussian step against the basis; returns novelty statistics (RingStats). |
| 5 | commit | H(t) is committed when the pass brings new bits or the ring basis changed; a basis change alone is a structural commit. A no-change pass commits nothing. |
| 6 | materialize | On demand, bits are lowered back to tokens (LUT-first, probabilistic restoration) for the host. |
-

The D/R/N decomposition is computed from $S(t)$ and $H(t-1)$ at every commit — the per-transition reading that later frames build on. A commit records `ewm-app::run_turn` → tip advance in `ewm-git`.

XOR arithmetic turns originals into a coordinate system

Work in a world where **adding means toggling**. For sets of bits, "add" is symmetric difference (XOR): a bit that appears twice cancels out — $x + x = 0$. That one rule, called **GF(2) arithmetic**, is the whole algebra.

The ring keeps a growing **reference list** of originals (a basis). Pushing a new original is one reduction step: XOR away the references it overlaps.

nothing left → already known

the original is exactly a combination of references —
nothing changes (residual = $\emptyset$, "in span")

something left → new direction

the leftover (residual) joins the reference list — the space
of combinations grows by one

The commit HLLSet is what a commit pushes; a base HLLSet is an entry in the reference list. Every update to the list is one of two kinds: extension and rotation.

WHAT AN UPDATE REPORTS — RingStats

extension — a new reference

the leftover becomes a new direction in the list

rotation — re-indexing

every reference sharing the new leading bit is re-expressed by XOR with the leftover, so leading bits stay distinct

rotation_count · rotation_mass

how many references were re-indexed · total bit change across the old list (= count × leftover)

residual · in_span · dimension

the unexplained leftover (a novelty reading) · whether anything changed · the current number of directions

The reference list only grows — so coordinates carry over exactly

The ring separates two roles: a **reference list** over every commit HLLSet ever pushed — it only ever grows — and a bounded **visibility window** of recent commits used as context. Eviction moves the window; it never removes a reference.

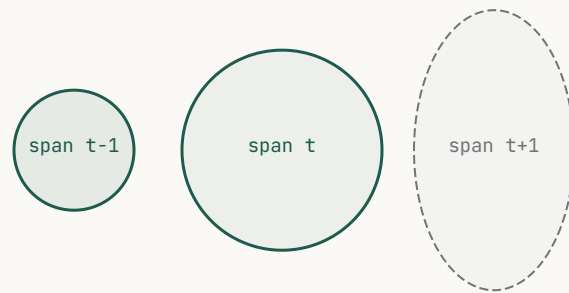
So the space of XOR-combinations (the **span**) only grows — and nothing once expressible ever stops being expressible: **once known, always known**.

Coordinates answer one question: **which references XOR together to make this set?** When the list grows, every old reference keeps a unique answer — a conversion table M carries old coordinates a to $a \cdot M$, **exactly**.

Every list version carries a **generation** stamp.

Exact transport (proposition). When a monotone list grows, every old reference has unique coordinates in the new one — anything expressible yesterday stays expressible today, and the table M carries the coordinates over exactly.

THE SPACE ONLY GROWS



The space never shrinks; every larger list keeps all old coordinates recoverable through the table M .

LIST GENERATIONS

every reading is valid only relative to the list generation it was taken in

What XOR arithmetic cannot see — and why that is useful

Toggling understands **combinations** — symmetric differences — but **not** intersections, unions, or differences. For two representable sets A and B, all three compounds are representable together or not at all, because each is one intersection away from a combination:

$$A \cup B = A \Delta B \Delta (A \cap B)$$

$$A \setminus B = A \Delta (A \cap B)$$

The intersection is the single obstruction. A lattice expression is representable exactly when its intersection content is. The test for any set X: reduce it against the reference list and check that nothing is left — $\text{res}(X) = \emptyset$.

This blind spot is a feature: the leftover measures **how far a set is from any combination of the current context** — an exact novelty signal, not a heuristic score.

MEASURED ON THE 10×10 CLIP

394/394

XOR combinations representable

0/394

$\cap, \cup, \setminus$ representable (this data)

394/394

pairs with equal leftover after reduction

ONE HONEST CAVEAT

The leftover is one representative of an equivalence class, not a canonical one: reduction stops at row-echelon form, so equivalent sets can leave different leftovers. The fact the tests rely on is exact — $\text{res}(X) = \emptyset \Leftrightarrow X$ can be built from the references.

The whole ring in four 8-bit vectors

XOR is toggling. $10110011 \Delta 01101110 = 11011101$ — matching bits cancel, differing bits stay.

1 · PUSH A — LIST IS EMPTY

A = 10110011

nothing to reduce against

list $\leftarrow$ [A]

extension — a first direction; span grows by one.

2 · PUSH B — NO OVERLAP AT THE FRONT

B = 01101110

leading bit not in the list

list $\leftarrow$ [A, B]

extension — a second, independent direction.

3 · PUSH C — REDUCE, SOMETHING LEFT

C = 11010101

C Δ A = 01100110

C Δ A Δ B = 00001000 $\rightarrow$ list $\leftarrow$ [A, B, D]

extension — the leftover D is C's new contribution; rotation re-indexes A and B.

4 · PUSH D — NOTHING LEFT

D = 00001000 = A Δ B Δ C

D Δ D = 00000000

coordinates(D) = {A, B, C} — span stays 3 of 8

already known — res = $\emptyset$ and in_span = true; a genuinely new E would leave res $\neq \emptyset$.

Same vectors, read two ways: D is old news (a combination); $11011101 = A \Delta B$ reduces to nothing as well — the list decides what counts as new.

Views, not stored compounds

Exact transport fixes what the store keeps. Only two kinds of node are materialized: **originals** (the IICA sketches of measured streams) and **base HLLSets** (the monotone ring basis) — each content-addressed as `h:<sha1>` and immutable.

Everything else — unions `S(t)`, `D/R/N`, residuals, soft keys, covers, projections — is a **view**: an expression plus its provenance (the ordered SHA-1 identifiers of its sources). A view stores no bits of its own.

Why: coordinates are basis-relative and the basis moves. A compound frozen as bits or coordinates in one base is stale in another — the recipe never is. The reader **recreates** the compound against the base current at read time; recreation is deterministic and idempotent because the sources are IICA.

Storage is proportional to **originals + base**, not to the number of compounds ever named — the operational form of "discover, do not create." Cached views are stamped with their basis generation and re-projected lazily, never wholesale.

A VIEW RECORD (ABBREVIATED)

```
{
  "view":    "p:9f2c1d...",
  "expr":    "union",
  "sources": ["h:1a7e...", "h:4b02...",
              "h:cd91..."],
  "basis_generation": 37,
  "materialized": false
}
```

STORED AS → READ AS

originals · base	bits, <code>h:<sha1></code> → itself
union, D/R/N	expression + provenance → recomputed
residual, soft key, cover	+ basis generation → recomputed; cache generation-stamped

Frames: the trajectory is a choice of lens

A **frame** is a named set of dimensions $D = (D_1, \dots, D_m)$; its coordinate map reads a node through BSS similarity, and $\varphi_D(X(t))$ over t is the **state trajectory** in that frame.

$$\varphi_D(X) = (BSS(X, D_1), \dots, BSS(X, D_m)), \quad BSS = |X \cap D_i| / |D_i|$$

Different frames are different readings of the same nodes: **D/R/N** is the temporal frame, the **ring basis** is the linear frame, joined perceptrons are the structural frame, a codebook gate is a per-user frame. Varying the frame changes what we see — not what the state is.

Basis frames also enable **time travel**: project the present into the first recorded basis; the spill $|X \setminus \text{cover}(F_0)|$ measures the error of reading the present through the past's lens.

FOUR TRAJECTORY SIGNALS

linear novelty — $\text{res}(X)$

the part no XOR of originals can express — a higher-order "new"

rotation — rotation_count

how many existing directions re-index through a new one; zero when residual is zero

bit novelty — $N_{\text{bit}} = X \setminus \text{cover}$

the part no span member can contain; cover identity
 $u \text{ basis} = u \text{ originals}$ holds exactly; $N_{\text{bit}} \subseteq \text{res}(X)$

basis frames — spill

every frame also projected into the first recorded basis, with the spill measuring past-lens error

EXACTNESS IS PART OF THE SPEC

change of basis: exact matrix · cover: exact · BSS transform:
 not linear — re-project, or climb the meet-order ladder ·
 minimal cover: unique support in span, greedy $H(d)$ otherwise

The basis is a rotating coordinate system

The ring's basis is not a fixed frame. Each commit appends a commit HLLSet (extension) and re-pivots some existing ones (rotation), so the basis **moves** — a node that does not change still changes its coordinates. On the 10×10 clip, **534 coordinates re-pivot over 100 commits** (mean 5.34 per commit): the frame moves on almost every step.

Each commit's **top state** $U(t)$ is read in the coordinates current when it arrived, $B(t-1)$ — the context it is measured against — yielding three exact objects. Arranged as a matrix with base coordinates on Y and commits on X, the trajectory becomes one readable picture.

THE COORDINATE-TIME MATRIX SHOWS THREE STRUCTURES

frontier	the white line of the span dimension — grows monotonically, bounds what exists yet
rows	born as coordinates appear; bright = stable context direction; dimming = rotated away
residual panel	spikes at transitions — a representation-independent event signal

DECOMPOSING THE TOP STATE $U(t)$

soft key — $w_i \in [0,1]$

$w_i = |U \cap B_i| / |B_i|$ — where the new state lands in the current coordinates; always defined

hard key — **coordinates** $c(t)$

the unique XOR combination with $U = \Delta_i c_i B_i$ — defined exactly when the residual is empty

residual — **the off-span part**

completes the exact decomposition $U = P \Delta \text{res}$ — what no current coordinate can express

On the clip: no top state fully inside the prior span (0/100) — every frame carries new drift. Mean soft-key 0.091, max 0.804: most coordinates barely touched, a few carry the frame.

Measured, not asymptotic: a 10×10 scene clip

The clip has known structure: 10 scenes × 10 frames, each frame a 200-token scene vocabulary plus 56 tokens of per-frame drift; scene cuts at frames 11, 21, ..., 91. Analyzed incrementally as frames arrive, and reproducible by the repo's notebooks (a bit-exact Python mirror of the ring) and the Rust test suite.

NOVELTY AT SCENE CUTS VS IN-SCENE DRIFT

signal (morphism)	cut mean	in-scene	separation
residual, windowed (μ_{ng})	782.9	271.4	2.9×
residual, monotone (μ_{flat})	612.6	328.3	1.9×
bit novelty N, monotone (μ_{flat})	99.0	26.4	3.75×

The windowed linear residual flags all 9/9 cuts (each above in-scene mean + 2σ); bit novelty is the sharpest signal on the flat morphism. The two morphisms are different measurements — scales are not comparable; the structural conclusions are morphism-agnostic.

REPRESENTABILITY, MEASURED

Δ always in span (394/394); $\cap \cup$ never (0/394) — the meet obstruction, as predicted. Change of basis 50 → 100 is sparse (density 0.038); all 50 old basis elements stay representable; the coordinate round-trip is the identity.

HEADLINE NUMBERS

9 / 9

scene cuts detected by the linear residual

3.75×

bit-novelty separation of cuts from in-scene drift

534

coordinates re-pivoted over 100 commits — the frame moves

442 → 12

minimal cover compresses a five-frame context of 442 bits into 12 basis elements, full coverage

The residual is not a heuristic score bolted onto a buffer — it is the distance from a node to the span of the context, computed exactly, incrementally, cheaply.

Where this sits — and what it costs

POSITIONING — WHAT THIS IS, WHAT IT IS NOT

SKETCHES

HLL, Bloom, min-wise estimate cardinality. We do not estimate — **the sketch is the state.**

CONTENT ADDRESSING

Merkle trees, Git, IPFS: immutable, hash-identified objects. We add a lattice the objects live on.

CRDTs

Join is set union on a grow-only set; idempotence comes from IICA, not from a protocol.

WORLD MODELS

JEPA, RAG and friends learn or retrieve. The side-car is orthogonal: it does not learn the world, it holds the state.

The claim is narrow and checkable: one shared lattice, exact comparisons, measurable novelty — **with the costs measured, not hidden.**

LIMITATIONS — STATED PLAINLY

- 01 No canonical basis.** Row-echelon, not fully reduced: bases are valid, not unique.
- 02 Remainder is a coset representative** — congruent sets can return different remainders.
- 03 Monotone growth costs memory.** The windowed mode trades exact replay for a bounded basis.
- 04 Windowed semantics weaken algebra.** Under μ_{ng} , union identities survive only as inclusions.
- 05 Recomputation is the price of correctness** — mitigated by generation-stamped caching, not eliminated.

None of these breaks the core guarantees: every commit is still idempotent, immutable and content-addressed, and the span test $\text{res}(X) = \emptyset$ stays exact.

ONE LATTICE · CHEAP TO ADVANCE · EXACT TO COMPARE · SAFE TO REPLAY

Any token-producing model can be given a state.

Cheap to advance — one set join and a ring push per commit; no retraining, no extra forward pass.

Exact to compare — D / R / N per commit with a conservation balance, not a similarity score.

Safe to replay — immutable, content-addressed commits; every state is re-derivable from the log.

Measurable novelty — residual, rotation count and bit novelty read off a monotone basis, cut by cut.

WHAT IS NEXT

Trajectory-stability study — how frames and rotations behave over long horizons.

Multi-world extension — many agents on one lattice, separated at the interpretation layer.

Closing the loop — memory plus a curiosity controller driving the host.

Scaled evaluation — longer horizons, larger vocabularies, more hosts.

Acknowledgment — this deck was prepared with the assistance of Kimi (Moonshot AI).

$H(t) = (S(t), H(t-1), D, R, N)$

github.com/alexmy21/ewm-state-machine

